

1. Opis problemu

Celem projektu jest stworzenie symulacji baseballowej, która przewiduje wynik uderzenia (single, double, triple, home run, inside-the-park home run lub out) na podstawie atrybutów zawodników oraz miejsca upadku piłki. Model wykorzystuje uproszczone statystyki zawodników takie jak: prędkość biegu pałkarza oraz prędkość i siłę rzutu outfielderów. Dodatkowo używa algorytm A* do wyznaczenia najszybszej ścieżki obrońcy do piłki. Porównanie czasu obrony z czasem biegu pałkarza pozwala odtworzyć realistyczne scenariusze znane z profesjonalnych gier sportowych, takich jak MLB The Show.

2. Opis danych

W projekcie wykorzystywane są dane opisujące:

Atrybuty pałkarzy

Pałkarz posiada jedną kluczową cechę wpływającą na przebieg akcji:

- **Speed** – prędkość biegu po bazach, wpływa na czas dotarcia do kolejnych baz

Atrybuty outfielderów (LF, CF, RF)

Każdy z trzech outfielderów posiada zestaw parametrów wpływających na czas reakcji obrony:

- **Speed** – prędkość biegu w kierunku piłki
- **Arm Strength** – prędkość rzutu do bazy

Dane przestrzenne

Boisko baseballowe jest reprezentowane jako siatka 2D (350x350), w której:

- home plate, 1st, 2nd i 3rd base mają stałe współrzędne
 - pozycje outfielderów są zdefiniowane na początku symulacji
 - punkt upadku piłki jest generowany losowo
-

3. Uzasadnienie wyboru algorytmu

Do obliczenia minimalnego czasu dotarcia zawodników do piłki zastosowano algorytm A^* , ponieważ jest on prosty w implementacji, dobrze rozumiały oraz bardzo efektywny w przeszukiwaniu siatek 2D.

Alternatywne algorytmy, takie jak Dijkstra czy BFS/DFS, nie były odpowiednie do tego typu projektu. BFS przeszukuje szeroko całą siatkę, co jest nieefektywne na dużym boisku, a DFS nie gwarantuje najkrótszej ścieżki, więc nie nadaje się do obliczania minimalnego czasu dotarcia do piłki. Dijkstra działa poprawnie, ale przeszukuje przestrzeń we wszystkich kierunkach, przez co jest wolniejszy od A^* .

W projekcie wykorzystano heurystykę Manhattan, która jest zgodna z ruchem po kratce (bez poruszania się po skosie), co dodatkowo upraszcza obliczenia i przyspiesza działanie algorytmu. Heurystyka euklidesowa była rozważana, jednak w praktyce okazała się bardziej kłopotliwa w implementacji, dlatego ostatecznie zdecydowano się pozostać przy prostszej i wystarczająco dokładnej heurystyce.

4. Krótki opis metody

Metoda wykorzystuje algorytm A^* do wyznaczenia najkrótszej trasy każdego outfieldera do piłki. Boisko jest traktowane jako graf, w którym każde pole ma swoich sąsiadów, a A^* sprawdza je, wybierając te prowadzące najbliżej celu na podstawie kosztu ruchu i heurystyki. Dla każdego obrońcy obliczana jest długość ścieżki i wynikający z niej czas dotarcia do piłki. Spośród wszystkich outfielderów wybierany jest ten, który ma najkrótszy dystans i czas. Równolegle liczony jest czas biegu pałkarza do kolejnych baz, a porównanie obu czasów pozwala określić, czy zdąży bezpiecznie dotrzeć przed zagranie obrony.

5. Opis przeprowadzonych testów

W projekcie przygotowano zestaw testów jednostkowych obejmujących trzy kluczowe moduły: pathfinding (A^*), logikę wyniku zagrania oraz fizykę czasu biegu zawodników. Celem testów było sprawdzenie poprawności działania algorytmów oraz ich zgodności z założeniami symulacji.

Testy A* weryfikują, czy algorytm poprawnie znajduje najkrótszą ścieżkę w małym, kontrolowanym grafie. Sprawdzane jest, czy zwrócona ścieżka zaczyna się w punkcie startowym, kończy w punkcie docelowym, ma dodatni koszt oraz zawiera co najmniej dwa węzły. Dzięki temu potwierdzono, że A* prawidłowo analizuje sąsiadów i wyznacza optymalną trasę.

Testy logiki gry sprawdzają funkcję odpowiedzialną za określenie wyniku zagrania. Dla różnych zestawów czasów biegu pałkarza i obrońcy testowane są wszystkie możliwe scenariusze. Każdy przypadek ma przygotowane dane wejściowe, które jednoznacznie prowadzą do oczekiwanego wyniku.

Testy fizyki obejmują generowanie czasów biegu pałkarza oraz obliczanie całkowitego czasu obrońcy. Sprawdzane jest, czy funkcje zwracają wszystkie wymagane klucze, czy wartości są dodatnie oraz czy czas rośnie wraz z dystansem. Dodatkowo testowane jest, czy większa odległość obrońcy zawsze skutkuje dłuższym czasem dotarcia.

6. Uzyskane wyniki

Wszystkie przygotowane testy jednostkowe zakończyły się wynikiem pozytywnym. Testy potwierdziły poprawne działanie kluczowych elementów symulacji. Algorytm A* zawsze zwracał poprawną ścieżkę i dodatni koszt, funkcja logiki gry zwróciła właściwe wyniki dla wszystkich scenariuszy, a testy fizyki potwierdziły poprawność generowanych czasów oraz ich zależność od dystansu. Otrzymane rezultaty wskazują, że wszystkie moduły działają zgodnie z założeniami projektu.

```
..... [100%]  
===== tests coverage =====  
----- coverage: platform win32, python 3.14.0-final-0 -----  


| Name               | Stmts | Miss | Cover |
|--------------------|-------|------|-------|
| app\drawing.py     | 27    | 27   | 0%    |
| app\field.py       | 33    | 33   | 0%    |
| app\logic.py       | 10    | 0    | 100%  |
| app\main.py        | 46    | 46   | 0%    |
| app\pathfinding.py | 46    | 5    | 89%   |
| app\physics.py     | 14    | 0    | 100%  |
| TOTAL              | 176   | 111  | 37%   |

  
Coverage XML written to file coverage.xml  
11 passed in 0.23s
```

7. Wnioski

Przeprowadzona implementacja oraz testy potwierdziły poprawność działania całego systemu. Algorytm A* prawidłowo wyznacza najkrótsze trasy obrońców, a moduły fizyki i logiki gry poprawnie obliczają czasy oraz wynik zagrania. Wszystkie testy jednostkowe zakończyły się pozytywnie, co potwierdza zgodność działania aplikacji z założeniami projektowymi. Ostatecznie system poprawnie symuluje sytuacje meczowe i umożliwia wiarygodne porównanie czasu reakcji obrony z czasem biegu pałkarza.